

Node.js 교과서



Node.js 교과서 개정2판



7장

7.1 데이터베이스

7.2 MySQL, 워크벤치 설치하기

7.3 데이터베이스, 테이블 생성하기

7.4 CRUD 작업하기

7.5 시퀀라이즈 사용하기

7.1 데이터베이스



1. 데이터베이스란

» 지금까지는 데이터를 서버 메모리에 저장했음

- 서버를 재시작하면 데이터도 사라져버림 -> 영구적으로 저장할 공간 필요

» MySQL 관계형 데이터베이스 사용

- 데이터베이스: 관련성을 가지며 중복이 없는 데이터들의 집합
- DBMS: 데이터베이스를 관리하는 시스템
- RDBMS: 관계형 데이터베이스를 관리하는 시스템
- 서버의 하드 디스크나 SSD 등의 저장 매체에 데이터를 저장
- 서버 종료 여부와 상관 없이 데이터를 계속 사용할 수 있음
- 여러 사람이 동시에 접근할 수 있고, 권한을 따로 줄 수 있음

▼ 그림 7-2 데이터베이스는 흔히 원기둥 세 개를 겹친 모양으로 표현합니다.



7.2 MySQL, 워크벤치 설치하기

1. 원도

» 공식 사이트(<https://dev.mysql.com/downloads/installer>)

▼ 그림 7-3 MySQL Installer 다운로드 화면

MySQL Community Downloads

MySQL Installer

General Availability (GA) Releases Archives

MySQL Installer 8.0.19

Select Operating System:
Microsoft Windows

Looking for previous GA versions?

Windows (x86, 32-bit), MSI Installer (mysql-installer-web-community-8.0.19.0.msi)	8.0.19	18.6M	Download
Windows (x86, 32-bit), MSI Installer (mysql-installer-community-8.0.19.0.msi)	8.0.19	398.9M	Download

MD5: 32043798cd229db46fddaa8ddc11ad81 | Signature

MD5: 1a882015dk7f658f20c4717663b6817c | Signature

We suggest that you use the MD5 checksums and GnuPG signatures to verify the integrity of the packages you download.

ORACLE © 2020, Oracle Corporation and/or its affiliates

Legal Policies | Your Privacy Rights | Terms of Use | Trademark Policy | Contributor Agreement | Cookie Preferences

▼ 그림 7-4 로그인하지 않아도 됩니다.

MySQL Community Downloads

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

[Login »](#)
using my Oracle Web account

[Sign Up »](#)
for an Oracle Web account

MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can sign up for a free account by clicking the Sign Up link and following the instructions.

[No thanks, just start my download.](#)

ORACLE © 2020, Oracle Corporation and/or its affiliates

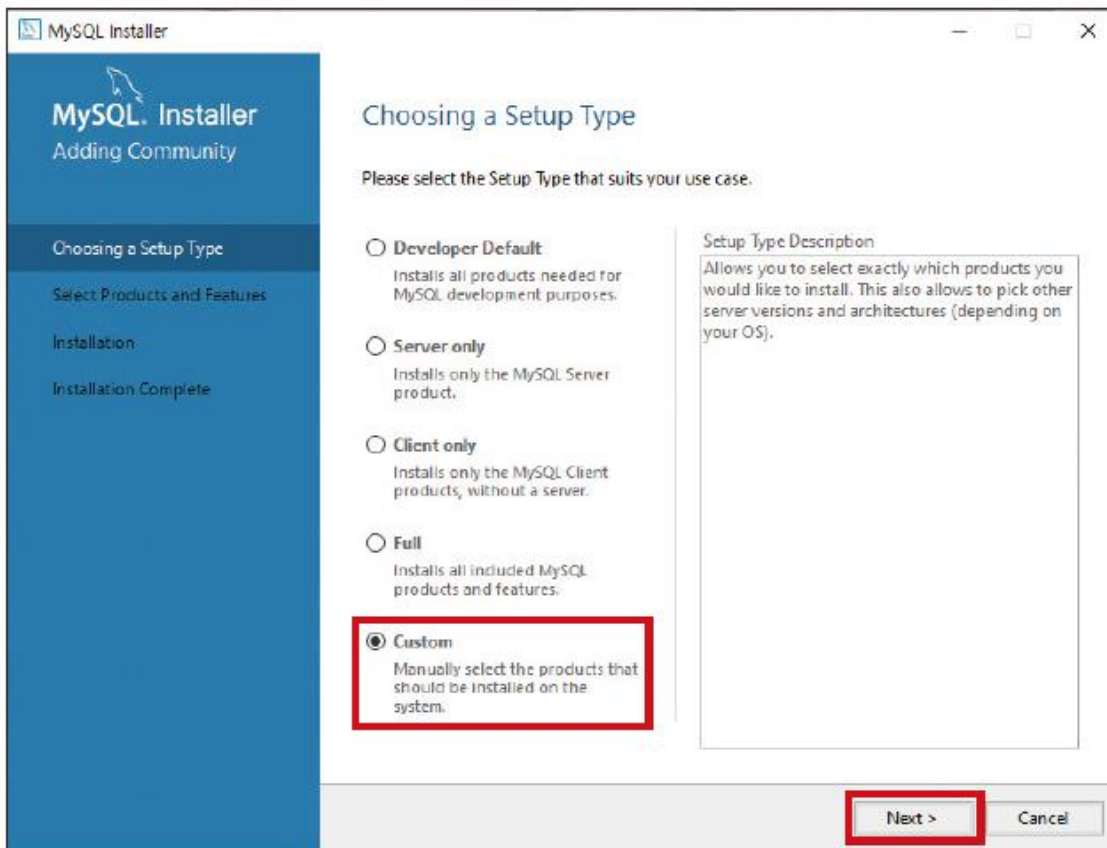
Legal Policies | Your Privacy Rights | Terms of Use | Trademark Policy | Contributor Agreement | Cookie Preferences

2. 윈도우 MySQL Installer

» 다운로드 받은 MySQL Installer 실행

- 설치 진행 중 Choosing a Setup Type 부분에서 Custom 선택

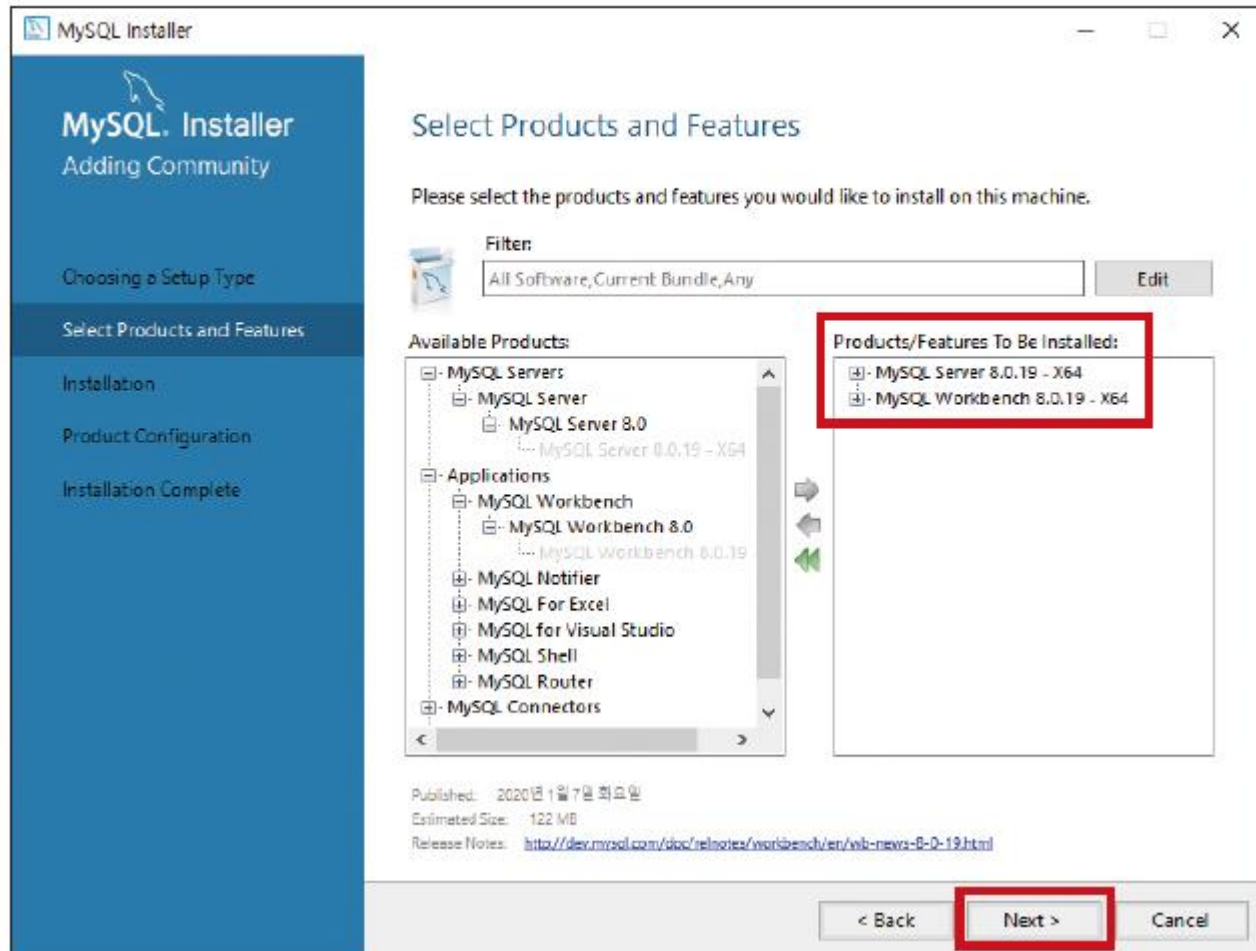
▼ 그림 7-5 Custom을 선택합니다.



3. Server와 Workbench 설치

» 자신의 운영체제(x86과 x64 중)에서 Server와 Workbench만 선택하여 설치

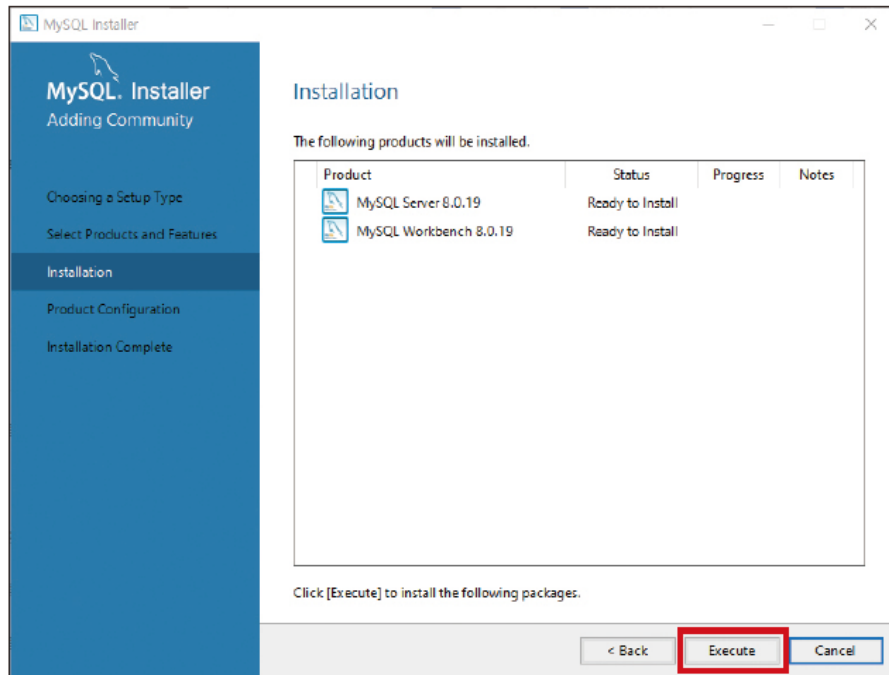
▼ 그림 7-6 MySQL Server와 MySQL Workbench 선택



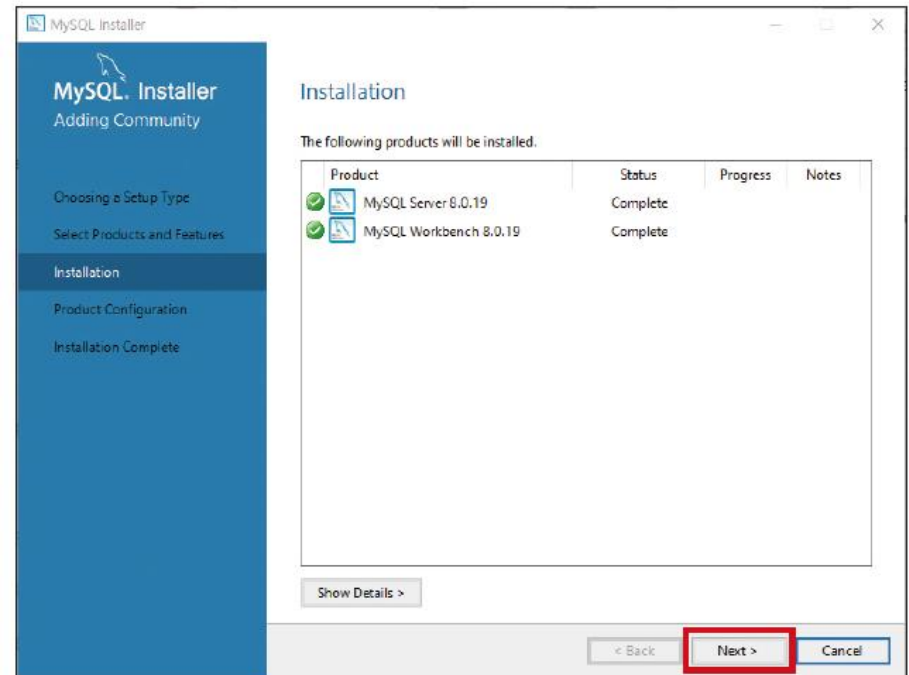
4. Execute

» Execute로 설치한 후 Next로 마무리

▼ 그림 7-7 설치 화면



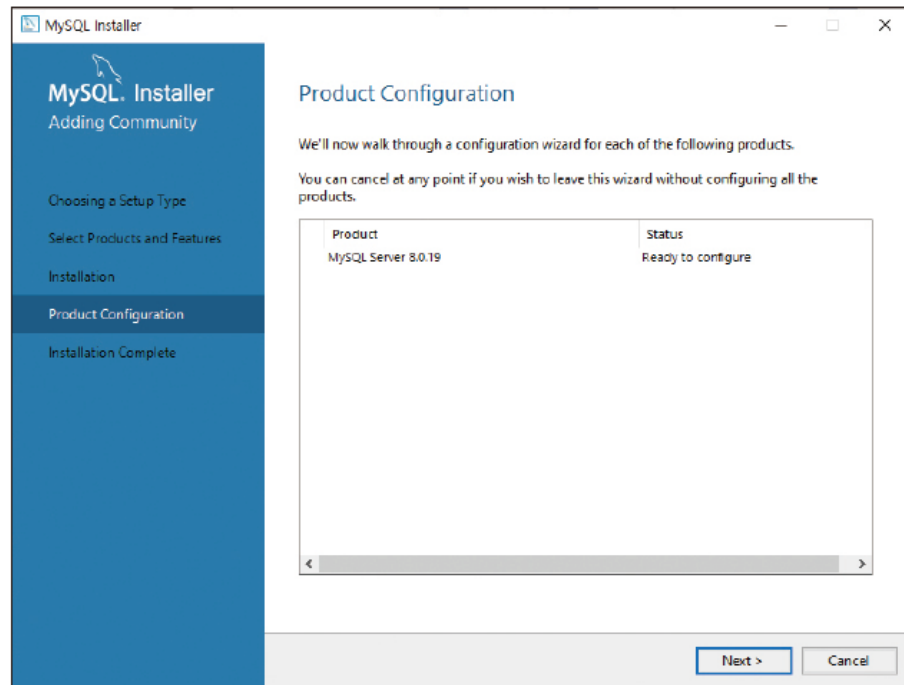
▼ 그림 7-8 설치 완료 화면



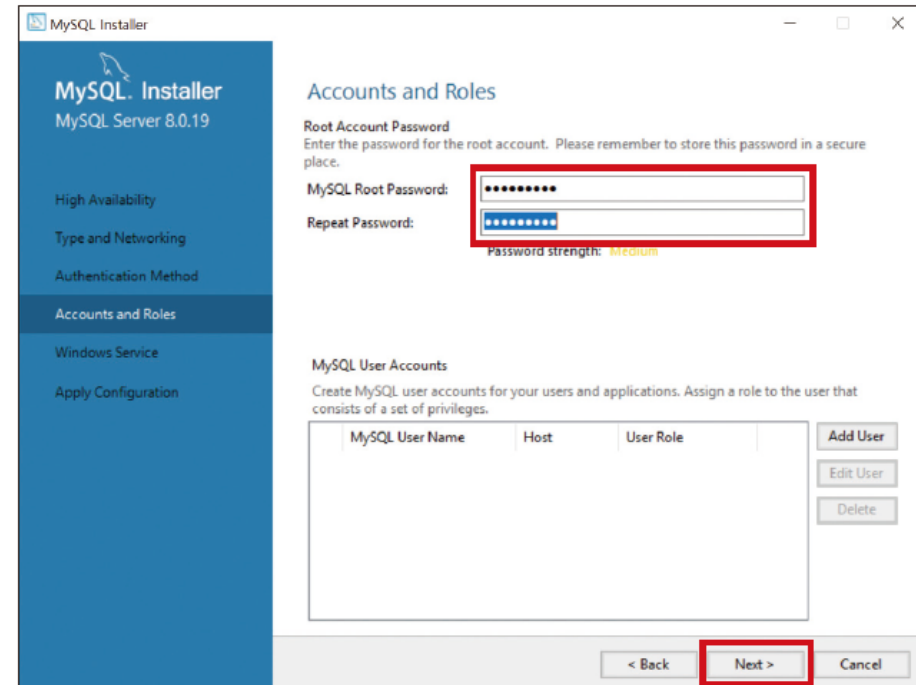
5. MySQL 기본 설정하기

» 계속 Next를 누르다가 비밀번호 설정하고 넘어가기

▼ 그림 7-9 MySQL 설정 화면



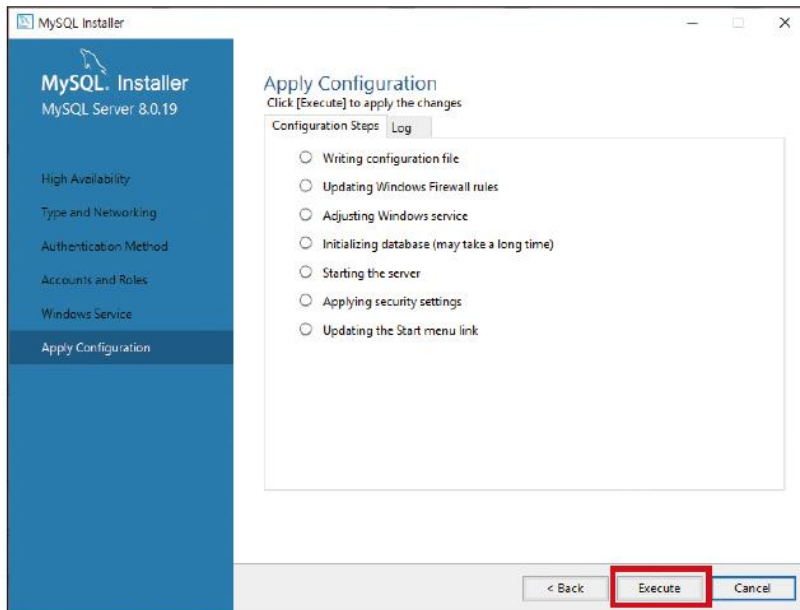
▼ 그림 7-10 root 비밀번호 설정 화면



6. 설정 적용하기

» Execute를 눌러 적용

♥ 그림 7-11 설정 반영 전 화면



Note ≡ Starting Server에서 막힌다면

Apply Configuration을 진행하는 도중 Starting Server에서 넘어가지 못하고 에러가 발생한다면 MySQL 설치 경로나 사용자 계정 이름, 컴퓨터 이름 등에 한글이 포함되어 있는 것은 아닌지 살펴봐야 합니다. 한글이 있다면 영어로 바꾸고 다시 시도해보세요. 프로그래밍 관련 프로그램을 설치할 때는 한글로 된 경로, 계정, 컴퓨터명 등을 사용하지 않는 것이 좋습니다.

7. MySQL 접속해보기

» 콘솔(CMD)에서 MySQL이 설치된 경로로 이동

- 기본 경로는 C:\Program Files\MySQL\MySQL Server 8.0\bin
- -h는 호스트, -u는 사용자 -p는 비밀번호 의미

콘솔

```
$ mysql -h localhost -u root -p
Enter password: [비밀번호 입력]
mysql>
```

- 프롬프트가 mysql>로 바뀐다면 성공
- 프롬프트를 종료하려면 exit 입력

콘솔

```
mysql> exit
Bye
```



8. macOS

» 홈브루(homebrew)를 통해 설치하는 게 편함

콘솔

```
$ /usr/bin/ruby -e "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install)"
```

» brew 설치 후 MySQL 설치

콘솔

```
$ brew install mysql  
$ brew services start mysql  
$ mysql_secure_installation
```

» root 비밀번호 설정 후, validate_password 플러그인을 설치하겠냐고 물으면 모두 n을 입력하고 엔터를 눌러 건너 뛴

- 실서버 운영 시에는 설정해주는 것이 좋음
- 윈도우와 같은 방법으로 MySQL에 접속



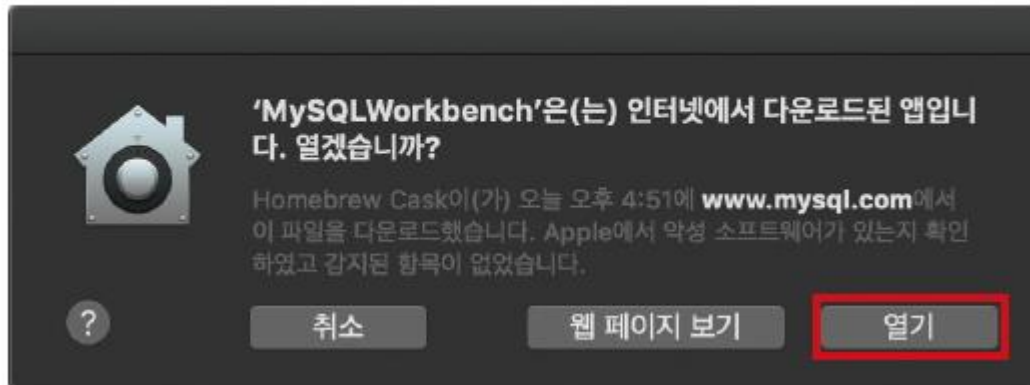
9. macOS 워크벤치 설치

» Homebrew로 다운로드

콘솔

```
$ brew cask install mysqlworkbench
```

▼ 그림 7-17 워크벤치 경고창



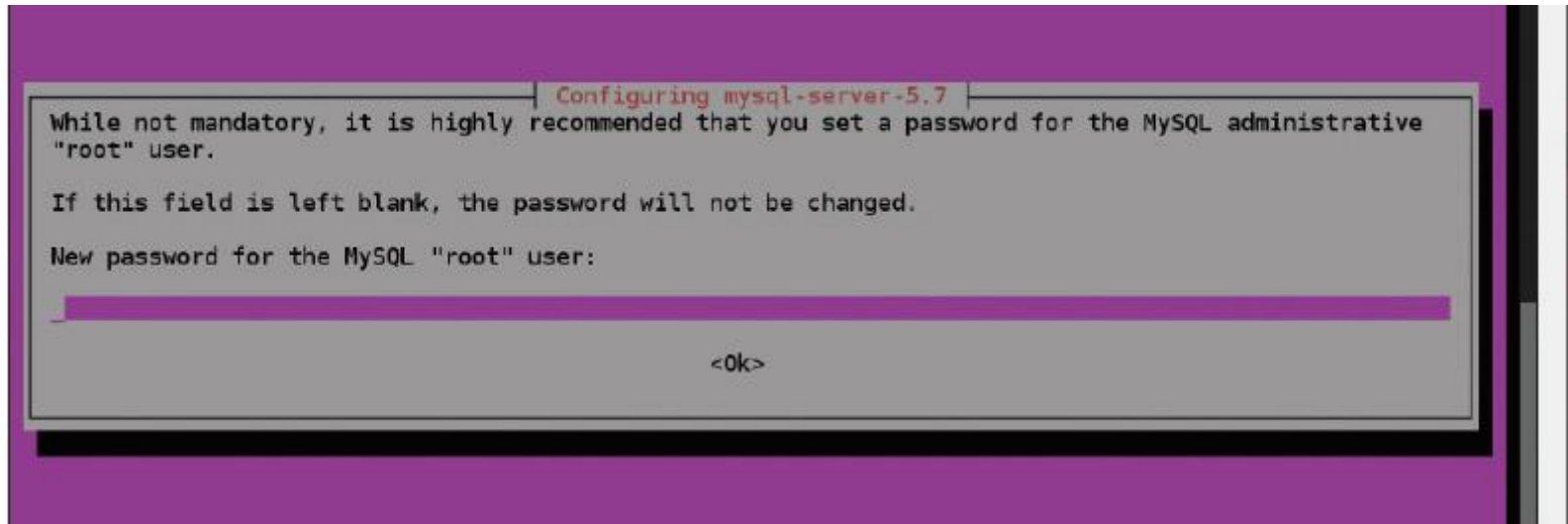
10. 리눅스(우분투)에 MySQL 설치

» GUI를 사용하지 않으므로 콘솔에 다음 명령어를 순서대로 입력

콘솔

```
$ sudo apt-get update  
$ sudo apt-get install -y mysql-server  
$ mysql_secure_installation
```

» root 비밀번호 설정



» 윈도우와 같은 방법으로 mysql에 접속

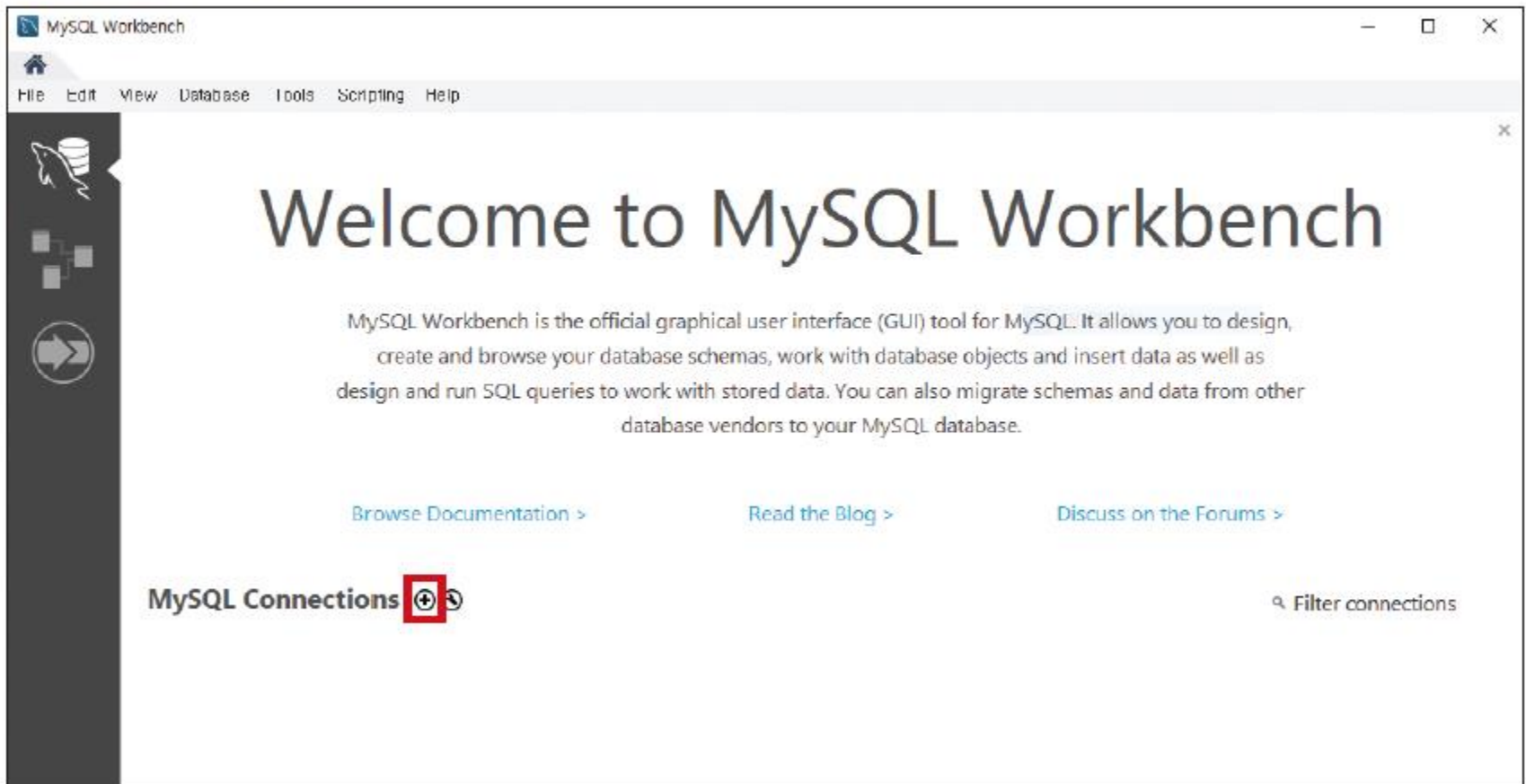
» 우분투의 경우는 워크벤치 대신 콘솔에서 작업

11. 윈도우, 맥 워크벤치에서 커넥션 생성

» 워크벤치 프로그램 실행

» MySQL Connections 옆에 + 모양 클릭

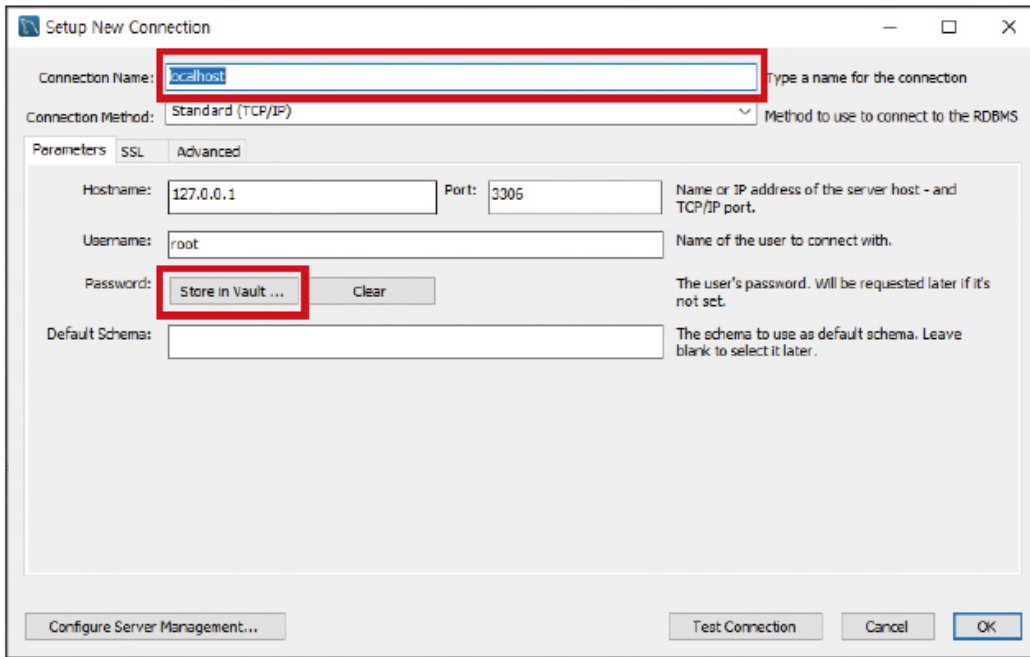
▼ 그림 7-18 워크벤치 실행 화면



12. MySQL 서버 주소 입력

» Connection Name에 localhost 적고 비밀번호 입력

▼ 그림 7-19 커넥션 생성 화면



The 'Setup New Connection' dialog box is shown. The 'Connection Name' field is highlighted with a red box and contains the text 'localhost'. The 'Connection Method' is set to 'Standard (TCP/IP)'. The 'Parameters' tab is selected, showing 'Hostname' as '127.0.0.1', 'Port' as '3306', 'Username' as 'root', and 'Password' as 'Store in Vault ...'. The 'Default Schema' field is empty. The 'OK' button is highlighted with a blue border.

▼ 그림 7-20 비밀번호 저장하기



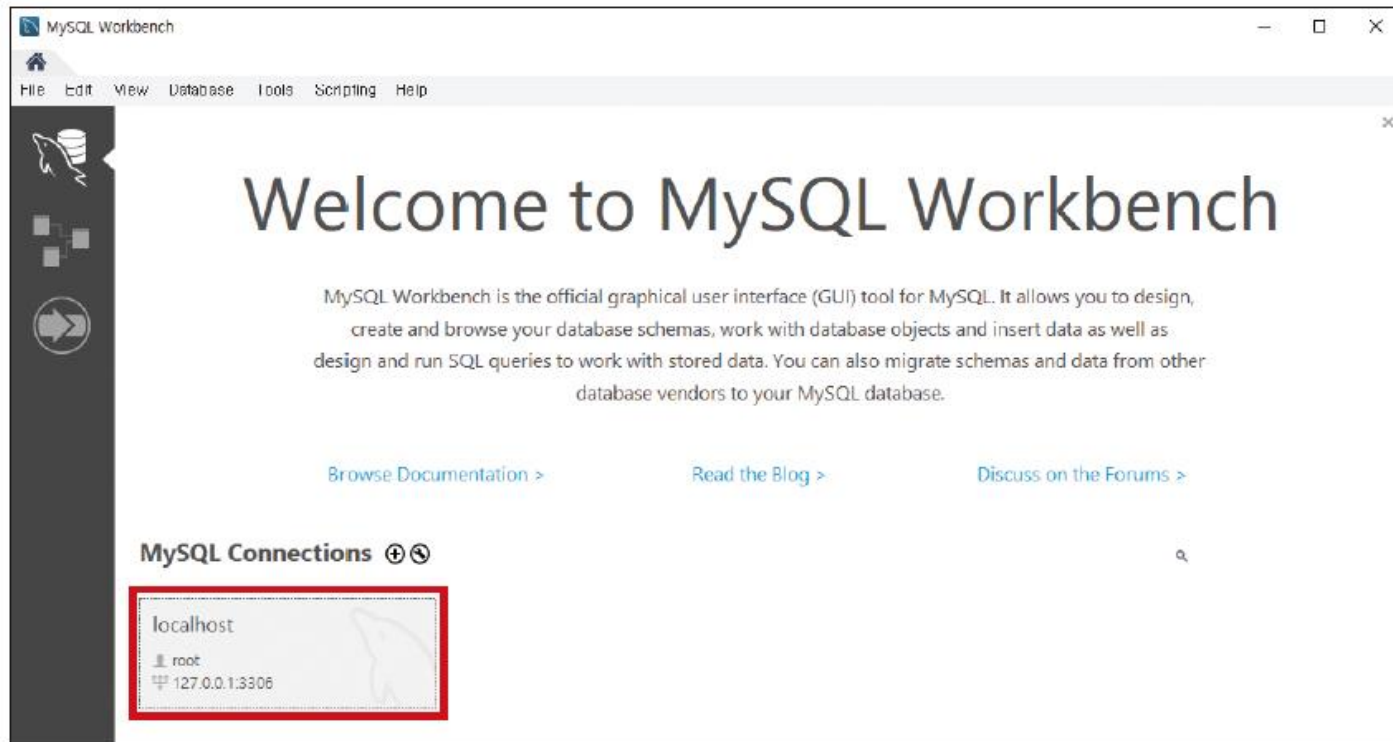
The 'Store Password For Connection' dialog box is shown. It prompts the user to enter a password for the service 'Mysql@127.0.0.1:3306' with the user 'root'. The 'Password' field is empty. The 'OK' button is highlighted with a red box.



13. localhost 서버 접속하기

» 아까 설정했던 비밀번호를 입력하여 접속

▼ 그림 7-21 localhost 접속하기



» 접속이 되었다면 성공

7.3 데이터베이스, 테이블 생성하기

1. 데이터베이스 생성하기

» 콘솔에서 MySQL 프롬프트에 접속

- **CREATE SCHEMA nodejs;**로 nodejs 데이터베이스 생성
- **use nodejs;**로 생성한 데이터베이스 선택

콘솔

```
mysql> CREATE SCHEMA `nodejs` DEFAULT CHARACTER SET utf8;  
Query OK, 1 row affected (0.01sec)  
mysql> use nodejs;  
Database changed
```

2. 테이블 생성하기

» MySQL 프롬프트에서 테이블 생성

- CREATE TABLE [데이터베이스명.테이블명]으로 테이블 생성
- 사용자 정보를 저장하는 테이블

콘솔

```
mysql> CREATE TABLE nodejs.comments (  
  -> id INT NOT NULL AUTO_INCREMENT,  
  -> commenter INT NOT NULL,  
  -> comment VARCHAR(100) NOT NULL,  
  -> created_at DATETIME NOT NULL DEFAULT now(),  
  -> PRIMARY KEY(id),  
  -> INDEX commenter_idx (commenter ASC),  
  -> CONSTRAINT commenter  
  -> FOREIGN KEY (commenter)  
  -> REFERENCES nodejs.users (id)  
  -> ON DELETE CASCADE  
  -> ON UPDATE CASCADE)  
  -> COMMENT = '댓글'  
  -> DEFAULT CHARSET=utf8mb4  
  -> ENGINE=InnoDB;
```

Query OK, 0 row affected (0.09 sec)



3. 컬럼과 로우

- » 나이, 결혼 여부, 성별같은 정보가 컬럼
- » 실제로 들어가는 데이터는 로우

Note ≡ 컬럼과 로우

▼ 그림 7-27 컬럼과 로우

→ 로우(row)

id	name	age	married
1	zero	24	false
2	nero	32	true
3	hero	28	false

↓ 컬럼(column)

4. 컬럼 옵션들

» id INT NOT NULL AUTO_INCREMENT

- 컬럼명 옆의 것들은 컬럼에 대한 옵션들
- INT: 정수 자료형(FLOAT, DOUBLE은 실수)
- VARCHAR: 문자열 자료형, 가변 길이(Char은 고정 길이)
- TEXT: 긴 문자열은 TEXT로 별도 저장
- DATETIME: 날짜 자료형 저장
- TINYINT: -128에서 127까지 저장하지만 여기서는 1 또는 0만 저장해 볼 값 표현
- NOT NULL: 빈 값은 받지 않는다는 뜻(NULL은 빈 값 허용)
- AUTO_INCREMENT: 숫자 자료형인 경우 다음 로우가 저장될 때 자동으로 1 증가
- UNSIGNED: 0과 양수만 허용
- ZEROFILL: 숫자의 자리 수가 고정된 경우 빈 자리에 0을 넣음
- DEFAULT now(): 날짜 컬럼의 기본값을 현재 시간으로

5. Primary Key, Unique Index

» PRIMARY KEY(id)

- id가 테이블에서 로우를 특정할 수 있게 해주는 고유한 값임을 의미
- 학번, 주민등록번호같은 개념

» UNIQUE INDEX name_UNIQUE (name ASC)

- 해당 컬럼(name)이 고유해야 함을 나타내는 옵션
- name_UNIQUE는 이 옵션의 이름(아무거나 다른 걸로 지어도 됨)
- ASC는 인덱스를 오름차순으로 저장함의 의미(내림차순은 DESC)

6. 테이블 옵션

- » COMMENT: 테이블에 대한 보충 설명(필수 아님)
- » DEFAULT CHARSET: utf8로 설정해야 한글이 입력됨
- » ENGINE: InnoDB 사용(이외에 MyISAM이 있음, 엔진별로 기능 차이 존재)

7. 테이블 생성되었나 확인하기

» DESC 테이블명

콘솔

```
mysql> DESC users;
```

▼ 그림 7-27 DESC 명령어 결과

```
mysql> DESC users;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(20)	NO	UNI	NULL	
age	int unsigned	NO		NULL	
married	tinyint(1)	NO		NULL	
comment	text	YES		NULL	
created_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

6 rows in set (0.00 sec)

» 테이블 삭제하기: DROP TABLE 테이블명

콘솔

```
mysql> DROP TABLE users;
```



8. 댓글 테이블 저장하기

» comments 테이블 생성

콘솔

```
mysql> CREATE TABLE nodejs.comments (  
  -> id INT NOT NULL AUTO_INCREMENT,  
  -> commenter INT NOT NULL,  
  -> comment VARCHAR(100) NOT NULL,  
  -> created_at DATETIME NOT NULL DEFAULT now(),  
  -> PRIMARY KEY(id),  
  -> INDEX commenter_idx (commenter ASC),  
  -> CONSTRAINT commenter  
  -> FOREIGN KEY (commenter)  
  -> REFERENCES nodejs.users (id)  
  -> ON DELETE CASCADE  
  -> ON UPDATE CASCADE)  
  -> COMMENT = '댓글'  
  -> DEFAULT CHARSET=utf8mb4  
  -> ENGINE=InnoDB;
```

Query OK, 0 row affected (0.09 sec)

9. 외래키(foreign key)

» 댓글 테이블은 사용자 테이블과 관계가 있음(사용자가 댓글을 달기 때문)

- 외래키를 두어 두 테이블이 관계가 있다는 것을 표시
- FOREIGN KEY (컬럼명) REFERENCES 데이터베이스.테이블명 (컬럼)
- FOREIGN KEY (commenter) REFERENCES nodejs.users (id)
- 댓글 테이블에는 commenter 컬럼이 생기고 사용자 테이블의 id값이 저장됨
- ON DELETE CASCADE, ON UPDATE CASCADE
- 사용자 테이블의 로우가 지워지고 수정될 때 댓글 테이블의 연관된 로우들도 같이 지워지고 수정됨
- 데이터를 일치시키기 위해 사용하는 옵션(CASCADE 대신 SET NULL과 NO ACTION도 있음)



10. 테이블 목록 보기

- SHOW TABLES;

콘솔

```
mysql> SHOW TABLES;
```

```
+-----+
```

```
| Tables_in_nodejs |
```

```
+-----+
```

```
| comments          |
```

```
| users              |
```

```
+-----+
```

```
2 rows in set (0.00 sec)
```

7.4 CRUD 작업하기



1. CRUD

» Create, Read, Update, Delete의 두문자어

- 데이터베이스에서 많이 하는 작업 4가지

▼ 그림 7-37 CRUD 작업





2. Create

» INSERT INTO 테이블 (컬럼명들) VALUES (값들)

콘솔

```
mysql> INSERT INTO nodejs.users (name, age, married, comment) VALUES ('zero', 24, 0, '자기소개1');
```

Query OK, 1 row affected (0.01 sec)

```
mysql> INSERT INTO nodejs.users (name, age, married, comment) VALUES ('nero', 32, 1, '자기소개2');
```

Query OK, 1 row affected (0.02 sec)

콘솔

```
mysql> INSERT INTO nodejs.comments (commenter, comment) VALUES (1, '안녕하세요. zero의 댓글입니다');
```

Query OK, 1 row affected (0.02 sec)



3. Read

» SELECT 컬럼 FROM 테이블명

- SELECT * 은 모든 컬럼을 선택한다는 의미

콘솔

```
mysql> SELECT * FROM nodejs.users;
```

```
+-----+-----+-----+-----+-----+-----+
| id | name | age | married | comment | created_at |
+-----+-----+-----+-----+-----+-----+
| 1 | zero | 24 | 0 | 자기소개1 | 2017-10-25 07:06:33 |
| 2 | nero | 32 | 1 | 자기소개2 | 2017-10-25 09:25:40 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

- 컬럼만 따로 추리는 것도 가능

콘솔

```
mysql> SELECT name, married FROM nodejs.users;
```

```
+-----+-----+
| name | married |
+-----+-----+
| zero | 0 |
| nero | 1 |
+-----+-----+
2 rows in set (0.00 sec)
```



4. Read 옵션들

» WHERE로 조건을 주어 선택 가능

- AND로 여러가지 조건을 동시에 만족하는 것을 찾음

콘솔

```
mysql> SELECT name, age FROM nodejs.users WHERE married = 1 AND age > 30;
```

```
+-----+-----+
```

```
| name | age |
```

```
+-----+-----+
```

```
| nero | 32 |
```

```
+-----+-----+
```

```
1 row in set (0.00 sec)
```

- OR로 여러가지 조건 중 하나 이상을 만족하는 것을 찾음

콘솔

```
mysql> SELECT id, name FROM nodejs.users WHERE married = 0 OR age > 30;
```

```
+----+-----+
```

```
| id | name |
```

```
+----+-----+
```

```
| 1 | zero |
```

```
| 2 | nero |
```

```
+----+-----+
```

```
2 rows in set (0.01 sec)
```



5. 정렬해서 찾기

» ORDER BY로 특정 컬럼 값 순서대로 정렬 가능

- DESC는 내림차순, ASC 오름차순

콘솔

```
mysql> SELECT id, name FROM nodejs.users ORDER BY age DESC;
```

```
+----+-----+
```

```
| id | name |
```

```
+----+-----+
```

```
|  2 | nero |
```

```
|  1 | zero |
```

```
+----+-----+
```

```
2 rows in set (0.01 sec)
```



6. LIMIT, OFFSET

» LIMIT으로 조회할 개수 제한

콘솔

```
mysql> SELECT id, name FROM nodejs.users ORDER BY age DESC LIMIT 1;
+----+-----+
| id | name |
+----+-----+
|  2 | nero |
+----+-----+
1 row in set (0.00 sec)
```

» OFFSET으로 앞의 로우들 스킵 가능(OFFSET 2면 세 번째 것부터 찾음)

콘솔

```
mysql> SELECT id, name FROM nodejs.users ORDER BY age DESC LIMIT 1 OFFSET 1;
+----+-----+
| id | name |
+----+-----+
|  1 | zero |
+----+-----+
1 row in set (0.00 sec)
```



7. Update

» 데이터베이스에 있는 데이터를 수정하는 작업

- UPDATE 테이블명 SET 컬럼=새값 WHERE 조건

콘솔

```
mysql> UPDATE nodejs.users SET comment = '바꿀 내용' WHERE id = 2;  
Query OK, 1 row affected (0.01 sec)  
Rows matched: 1  Changed: 1  Warnings: 0
```



8. Delete

» 데이터베이스에 있는 데이터를 삭제하는 작업

- DELETE FROM 테이블명 WHERE 조건

콘솔

```
mysql> DELETE FROM nodejs.users WHERE id = 2;  
Query OK, 1 row affected (0.00 sec)
```

7.5 시퀀셜라이즈 사용하기



1. 시퀀라이즈 ORM

- » MySQL 작업을 쉽게 할 수 있도록 도와주는 라이브러리
 - ORM: Object Relational Mapping: 객체와 데이터를 매핑(1대1 짝지음)
 - MySQL 외에도 다른 RDB(Maria, Postgre, SQLite, MSSQL)와도 호환됨
 - 자바스크립트 문법으로 데이터베이스 조작 가능
- » 시퀀라이즈 예제는 <https://github.com/zerocho/nodejs-book/tree/master/ch7/7.6/learn-sequelize>
- 프로젝트 세팅 후, 콘솔을 통해 경로로 이동한 후 package.json 작성

package.json

```
{  
  "name": "learn-sequelize",  
  "version": "0.0.1",  
  "description": "시퀀라이즈를 배우자",  
  "main": "app.js",  
  "scripts": {  
    "start": "nodemon app"  
  },  
  "author": "ZeroCho",  
  "license": "MIT"  
}
```



2. 시퀀라이즈 CLI 사용하기

» 시퀀라이즈 명령어 사용하기 위해 sequelize-cli 설치

- mysql2는 MySQL DB가 아닌 드라이버(Node.js와 MySQL을 이어주는 역할)

콘솔

```
$ npm i express morgan nunjucks sequelize sequelize-cli mysql2
$ npm i -D nodemon
```

» npx sequelize init으로 시퀀라이즈 구조 생성

콘솔

```
$ npx sequelize init
Sequelize CLI [Node: 14.0.0, CLI: 6.1.0, ORM: 6.2.3]
Created "config\config.json"
Successfully created models folder at ...
Successfully created migrations folder at ...
Successfully created seeders folder at ...
```



3. modes/index.js 수정

» 다음과 같이 수정

- require(../config/config) 설정 로딩
- new Sequelize(옵션들...)로 DB와 연결 가능

models/index.js

```
const Sequelize = require('sequelize');
```

```
const env = process.env.NODE_ENV || 'development';
```

```
const config = require('../config/config')[env];
```

```
const db = {};
```

```
const sequelize = new Sequelize(config.database, config.username, config.password,  
➡ config);
```

```
db.sequelize = sequelize;
```

```
module.exports = db;
```



4. MySQL 연결하기

» app.js 작성

- sequelize.sync로 연결

app.js

```
const express = require('express');
const path = require('path');
const morgan = require('morgan');
const nunjucks = require('nunjucks');

const { sequelize } = require('./models');

const app = express();
app.set('port', process.env.PORT || 3001);
app.set('view engine', 'html');
nunjucks.configure('views', {
  express: app,
  watch: true,
});
sequelize.sync({ force: false })
  .then(() => {
    console.log('데이터베이스 연결 성공');
  })
  .catch((err) => {
    console.error(err);
  });
```

```
app.use(morgan('dev'));
app.use(express.static(path.join(__dirname, 'public')));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

app.use((req, res, next) => {
  const error = new Error(`${req.method} ${req.url} 라우터가 없습니다.`);
  error.status = 404;
  next(error);
});

app.use((err, req, res, next) => {
  res.locals.message = err.message;
  res.locals.error = process.env.NODE_ENV !== 'production' ? err : {};
  res.status(err.status || 500);
  res.render('error');
});

app.listen(app.get('port'), () => {
  console.log(app.get('port'), '번 포트에서 대기 중');
});
```



5. config.json 설정하기

» DB 연결 정보를 넣기

config/config.json

```
{  
  "development": {  
    "username": "root",  
    "password": "[root 비밀번호]",  
    "database": "nodejs",  
    "host": "127.0.0.1",  
    "dialect": "mysql"  
  },  
  ...  
}
```



6. 연결 테스트하기

» npm start로 실행해서 SELECT 1+1 AS RESULT가 나오면 연결 성공

콘솔

```
$ npm start
> learn-sequelize@0.0.1 start C:\Users\zerocho\learn-sequelize
> nodemon app
```

```
[nodemon] 2.0.3
[nodemon] to restart at any time, enter `rs`
[nodemon] watching dir(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node app.js`
```

3001 번 포트에서 대기 중

Executing (default): SELECT 1+1 AS result

데이터베이스 연결 성공



7. 모델 생성하기

» 테이블에 대응되는 시퀀라이즈 모델 생성

models/user.js

```
const Sequelize = require('sequelize');

module.exports = class User extends Sequelize.Model {
  static init(sequelize) {
    return super.init({
      name: {
        type: Sequelize.STRING(20),
        allowNull: false,
        unique: true,
      },
      age: {
        type: Sequelize.INTEGER.UNSIGNED,
        allowNull: false,
      },
      married: {
        type: Sequelize.BOOLEAN,
        allowNull: false,
      },
      comment: {
        type: Sequelize.TEXT,
        allowNull: true,
      },
      created_at: {
        type: Sequelize.DATE,
        allowNull: false,
        defaultValue: Sequelize.NOW,
      },
    }, {
```

```
    }, {
      sequelize,
      timestamps: false,
      underscored: false,
      modelName: 'User',
      tableName: 'users',
      paranoid: false,
      charset: 'utf8',
      collate: 'utf8_general_ci',
    });
  }
  static associate(db) {}
};
```

8. 모델 옵션들

» 시퀀라이즈 모델의 자료형은 MySQL의 자료형과 조금 다름

♥ 표 7-1 MySQL과 시퀀라이즈의 비교

MySQL	시퀀라이즈
VARCHAR(100)	STRING(100)
INT	INTEGER
TINYINT	BOOLEAN
DATETIME	DATE
INT UNSIGNED	INTEGER.UNSIGNED
NOT NULL	allowNull: false
UNIQUE	unique: true
DEFAULT now()	defaultValue: Sequelize.NOW

» define 메서드의 세 번째 인자는 테이블 이름

- timestamps: true면 createdAt(생성 시간), updatedAt(수정 시간) 컬럼을 자동으로 만듦
- 예제에서는 직접 created_at 컬럼을 만들었으므로 false로 함
- paranoid 옵션은 true면 deleteAt(삭제 시간) 컬럼을 만듦, 로우 복구를 위해 완전히 삭제하지 않고 deleteAt에 표시해둠
- underscored 옵션은 캐멀케이스로 생성되는 컬럼을 스네이크케이스로 생성
- modelName은 모델 이름, tableName 옵션은 테이블 이름을 설정
- charset과 collate는 한글 설정을 위해 필요(이모티콘 넣으려면 utf8mb4로)



9. 댓글 모델 생성하기

» comment.js 생성

models/comment.js

```
const Sequelize = require('sequelize');
```

```
module.exports = class Comment extends Sequelize.Model {  
  static init(sequelize) {  
    return super.init({  
      comment: {  
        type: Sequelize.STRING(100),  
        allowNull: false,  
      },  
      created_at: {  

```

```
        type: Sequelize.DATE,  
        allowNull: true,  
        defaultValue: Sequelize.NOW,  
      },  
    }, {  
      sequelize,  
      timestamps: false,  
      modelName: 'Comment',  
      tableName: 'comments',  
      paranoid: false,  
      charset: 'utf8mb4',  
      collate: 'utf8mb4_general_ci',  
    });  
  }  
}
```

```
  static associate(db) {}  
};
```



10. 댓글 모델 활성화하기

» index.js에 모델 연결

- init으로 sequelize와 연결
- associate로 관계 설정

models/index.js

```
const Sequelize = require('sequelize');
const User = require('./user');
const Comment = require('./comment');
...
db.sequelize = sequelize;

db.User = User;
db.Comment = Comment;

User.init(sequelize);
Comment.init(sequelize);

User.associate(db);
Comment.associate(db);

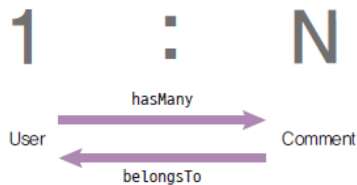
module.exports = db;
```



11. 관계 정의하기

» users 모델과 comments 모델 간의 관계를 정의

- 1:N 관계 (사용자 한 명이 댓글 여러 개 작성)
- 시퀀라이즈에서는 1:N 관계를 hasMany로 표현(사용자.hasMany(댓글))
- 반대의 입장에서는 belongsTo(댓글.belongsTo(사용자))
- belongsTo가 있는 테이블에 컬럼이 생김(댓글 테이블에 commenter 컬럼)



모델 각각의 static associate 메서드에 넣습니다.

models/user.js

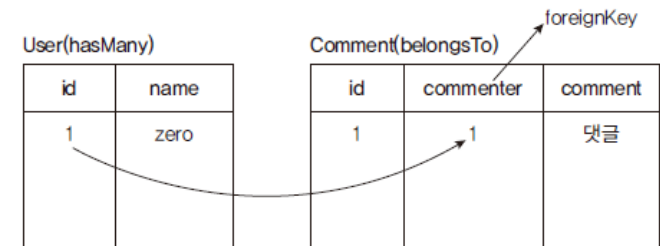
```

...
static associate(db) {
  db.User.hasMany(db.Comment, { foreignKey: 'commenter', sourceKey: 'id' });
}
};
  
```

models/comment.js

```

...
static associate(db) {
  db.Comment.belongsTo(db.User, { foreignKey: 'commenter', targetKey: 'id' });
}
};
  
```



commenter는 foreignKey
User의 id는 hasMany의 sourceKey이자
belongsTo의 targetKey



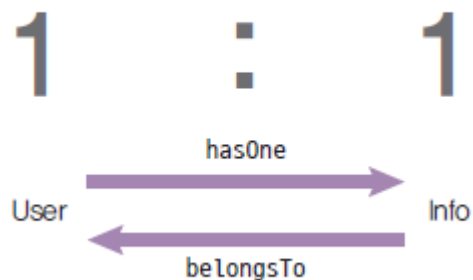
12. 1대1 관계

» 1대1 관계

- 예) 사용자 테이블과 사용자 정보 테이블

```
db.User.hasOne(db.Info, { foreignKey: 'UserId', sourceKey: 'id' });  
db.Info.belongsTo(db.User, { foreignKey: 'UserId', targetKey: 'id' });
```

▼ 그림 7-56 1:1 관계



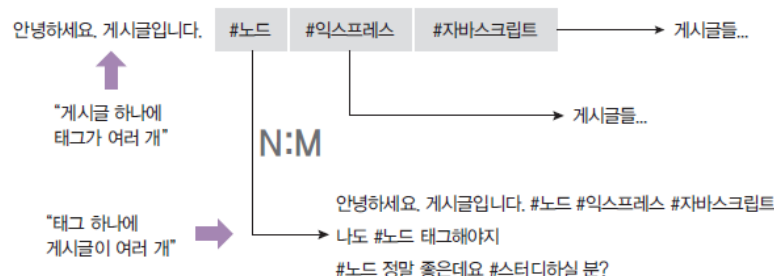


13. N대M 관계

» 다대다 관계

- 예) 게시물과 해시태그 테이블
- 하나의 게시물이 여러 개의 해시태그를 가질 수 있고 하나의 해시태그가 여러 개의 게시물을 가질 수 있음

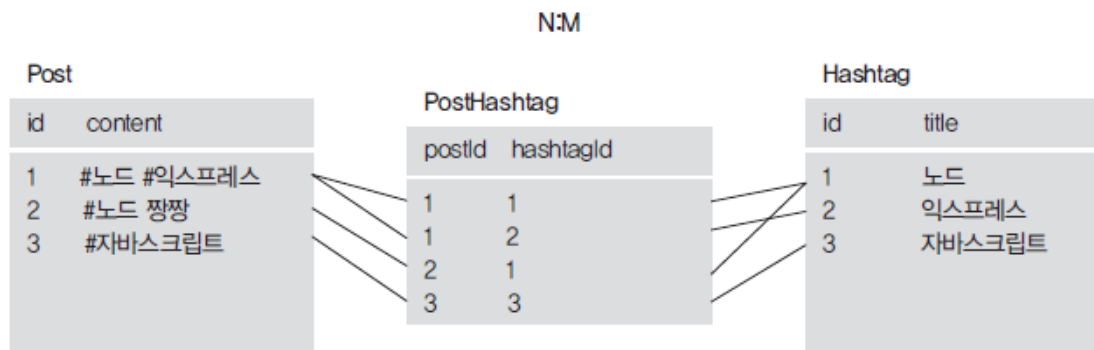
▼ 그림 7-57 N:M 관계



- DB 특성상 다대다 관계는 중간 테이블이 생김

```
db.Post.belongsToMany(db.Hashtag, { through: 'PostHashtag' });
db.Hashtag.belongsToMany(db.Post, { through: 'PostHashtag' });
```

▼ 그림 7-58 N:M 관계 테이블





14. 시퀀라이즈 쿼리 알아보기

» 위 줄이 SQL문, 아랫 줄은 시퀀라이즈 쿼리(자바스크립트)

```
INSERT INTO nodejs.users (name, age, married, comment) VALUES ('zero', 24, 0,  
➡ '자기소개1');
```

```
const { User } = require('../models');  
User.create({  
  name: 'zero',  
  age: 24,  
  married: false,  
  comment: '자기소개1',  
});
```

```
SELECT * FROM nodejs.users;  
User.findAll({});
```

```
SELECT name, married FROM nodejs.users;  
User.findAll({  
  attributes: ['name', 'married'],  
});
```



15. 시퀀라이즈 쿼리 알아보기

- » 윗 줄이 SQL문, 아랫 줄은 시퀀라이즈 쿼리(자바스크립트)
- 특수한 기능들인 경우 Sequelize.Op의 연산자 사용(gt, or 등)

```
SELECT name, age FROM nodejs.users WHERE married = 1 AND age > 30;
```

```
const { Op } = require('sequelize');
const { User } = require('../models');
User.findAll({
  attributes: ['name', 'age'],
  where: {
    married: 1,
    age: { [Op.gt]: 30 },
  },
});
```

```
SELECT id, name FROM users WHERE married = 0 OR age > 30;
```

```
const { Op } = require('sequelize');
const { User } = require('../models');
User.findAll({
  attributes: ['id', 'name'],
  where: {
    [Op.or]: [{ married: 0 }, { age: { [Op.gt]: 30 } }],
  },
});
```



16. 시퀀라이즈 쿼리 알아보기

» 윗 줄이 SQL문, 아랫 줄은 시퀀라이즈 쿼리(자바스크립트)

```
SELECT id, name FROM users ORDER BY age DESC;
```

```
User.findAll({  
  attributes: ['id', 'name'],  
  order: [['age', 'DESC']],  
});
```

```
SELECT id, name FROM users ORDER BY age DESC LIMIT 1;
```

```
User.findAll({  
  attributes: ['id', 'name'],  
  order: [['age', 'DESC']],  
  limit: 1,  
});
```

```
SELECT id, name FROM users ORDER BY age DESC LIMIT 1 OFFSET 1;
```

```
User.findAll({  
  attributes: ['id', 'name'],  
  order: ['age', 'DESC'],  
  limit: 1,  
  offset: 1,  
});
```



17. 시퀀라이즈 쿼리 알아보기

» 수정

```
UPDATE nodejs.users SET comment = '바꿀 내용' WHERE id = 2;  
User.update({  
  comment: '바꿀 내용',  
}, {  
  where: { id: 2 },  
});
```

» 삭제

```
DELETE FROM nodejs.users WHERE id = 2;  
User.destory({  
  where: { id: 2 },  
});
```



18. 관계 쿼리

» 결괏값이 자바스크립트 객체임

```
const user = await User.findOne({});  
console.log(user.nick); // 사용자 닉네임
```

» include로 JOIN 과 비슷한 기능 수행 가능(관계 있는 것 엮을 수 있음)

```
const user = await User.findOne({  
  include: [{  
    model: Comment,  
  }]  
});  
console.log(user.Comments); // 사용자 댓글
```

» 다대다 모델은 다음과 같이 접근 가능

```
db.sequelize.models.PostHashtag
```



19. 관계 쿼리

» get+모델명으로 관계 있는 데이터 로딩 가능

```
const user = await User.findOne({});  
const comments = await user.getComments();  
console.log(comments); // 사용자 댓글
```

» as로 모델명 변경 가능

```
// 관계를 설정할 때 as로 등록  
db.User.hasMany(db.Comment, { foreignKey: 'commenter', sourceKey: 'id', as: 'Answers'  
➡ });  
// 쿼리할 때는  
const user = await User.findOne({});  
const comments = await user.getAnswers();  
console.log(comments); // 사용자 댓글
```



20. 관계 쿼리

» include나 관계 쿼리 메서드에도 where나 attributes

```
const user = await User.findOne({
  include: [{
    model: Comment,
    where: {
      id: 1,
    },
    attributes: ['id'],
  }]
});
// 또는
const comments = await user.getComments({
  where: {
    id: 1,
  },
  attributes: ['id'],
});
```

» 생성 쿼리

```
const user = await User.findOne({});
const comment = await Comment.create();
await user.addComment(comment);
// 또는
await user.addComment(comment.id);
```



21. 관계 쿼리

» 여러 개를 추가할 때는 배열로 추가 가능

```
const user = await User.findOne({});  
const comment1 = await Comment.create();  
const comment2 = await Comment.create();  
await user.addComment([comment1, comment2]);
```

» 수정은 set+모델명, 삭제는 remove+모델명



22. raw 쿼리

» 직접 SQL을 쓸 수 있음

```
const [result, metadata] = await sequelize.query('SELECT * from comments');  
console.log(result);
```



23. 쿼리 수행하기

» <https://github.com/zerocho/nodejs-book/tree/master/ch7/7.6/learn-sequelize> 프론트 코드 복사

» 프론트 코드보다는 서버 코드 위주로 보기

- 프론트 코드는 서버에 요청을 보내는 AJAX 요청 위주로만 파악

routes/index.js

```
const express = require('express');
const User = require('../models/user');

const router = express.Router();

router.get('/', async (req, res, next) => {
  try {
    const users = await User.findAll();
    res.render('sequelize', { users });
  } catch (err) {
    console.error(err);
    next(err);
  }
}

module.exports = router;
```



24. 쿼리 수행하기

» users 라우터

- get, post, delete, patch 같은 요청에 대한 라우터 연결
- 데이터는 JSON 형식으로 응답
- comments 라우터도 마찬가지로

routes/users.js

```
const express = require('express');
const User = require('../models/user');
const Comment = require('../models/comment');

const router = express.Router();

router.route('/')
  .get(async (req, res, next) => {
    try {
      const users = await User.findAll();
      res.json(users);
    } catch (err) {
      console.error(err);
      next(err);
    }
  })
  .post(async (req, res, next) => {
    try {
      const user = await User.create({
        name: req.body.name,
        age: req.body.age,
        married: req.body.married,
      });
      console.log(user);
      res.status(201).json(user);
    } catch (err) {
      console.error(err);
      next(err);
    }
  })
```

```

  }
});

router.get('/:id/comments', async (req, res, next) => {
  try {
    const comments = await Comment.findAll({
      include: {
        model: User,
        where: { id: req.params.id },
      },
    });
    console.log(comments);
    res.json(comments);
  } catch (err) {
    console.error(err);
    next(err);
  }
});

module.exports = router;
```



25. 쿼리 수행하기

» comments 라우터

routes/comments.js

```
const express = require('express');
const { User, Comment } = require('../models');

const router = express.Router();

router.post('/', async (req, res, next) => {
  try {
    const comment = await Comment.create({
      commenter: req.body.id,
      comment: req.body.comment,
    });
    console.log(comment);
    res.status(201).json(comment);
  } catch (err) {
    console.error(err);
    next(err);
  }
});
```

```
router.route('/:id')
  .patch(async (req, res, next) => {
    try {
      const result = await Comment.update({
        comment: req.body.comment,
      }, {
        where: { id: req.params.id },
      });
      res.json(result);
    } catch (err) {
      console.error(err);
      next(err);
    }
  })
  .delete(async (req, res, next) => {
    try {
      const result = await Comment.destroy({ where: { id: req.params.id } });
      res.json(result);
    } catch (err) {
      console.error(err);
      next(err);
    }
  });
```

module.exports = router;



26. 서버 접속하기

» npm start로 서버 시작

- localhost:3000 으로 접속하면 시퀀라이즈가 수행하는 SQL문이 콘솔에 찍힘

콘솔

```
Executing (default): SELECT `id`, `name`, `age`, `married`, `comment`, `created_at` FROM
`users` AS `users`;
```

// 이하 생략

▼ 그림 7-59 접속 화면

사용자 등록

☐ 결혼 여부

아이디	이름	나이	결혼여부
1	zero	24	미혼

댓글 등록

아이디	작성자	댓글	수정	삭제
-----	-----	----	----	----



27. 서버 접속하기

» 글 등록/수정/삭제 해보기

▼ 그림 7-61 사용자 이름 클릭 시 화면

사용자 등록

이름

나이

☐ 결혼 여부

등록

아이디	이름	나이	결혼여부
1	zero	24	미혼

댓글 등록

사용자 아이디

댓글

등록

아이디	작성자	댓글	수정	삭제
1	zero	안녕하세요. zero의 댓글입니다.	수정	삭제

▼ 그림 7-62 nero 사용자 등록과 zero 댓글 작성 후 화면

사용자 등록

이름

나이

☐ 결혼 여부

등록

아이디	이름	나이	결혼여부
1	zero	24	미혼
4	nero	32	기혼

댓글 등록

사용자 아이디

댓글

등록

아이디	작성자	댓글	수정	삭제
1	zero	수정한 댓글입니다!	수정	삭제
2	zero	댓글을 등록합니다.	수정	삭제